FORMAL LOGIC

ANDREW WOOD

Contents

1.	Basic Definitions	1
2.	Classical Propositional Logic	2
3.	Natural Deduction	2
4.	Adequacy of the Propositional Rules	5
5.	First Order Logic (Predicate Logic)	7
6.	Interpretations and Adequacy of Predicate Logic	9
7.	Peano Arithmetic	11
8.	Kripke Models	12
9.	Applications	15
10.	References	16

1. Basic Definitions

Definition 1.1. An *argument* is a set of declarative sentences (propositions), one of which is the conclusion, while the rest are premises.

Definition 1.2. A *case* is an arrangement of truth values (True, False) to atomic propositional values.

Definition 1.3. An argument is *valid* if its conclusion is true and every case in which the premises are true.

Definition 1.4. An argument is *satisfiable* if there exists an interpretation for which the argument is true.

Definition 1.5. A *counter example* to an argument is a case in which the premises are true, but the conclusion is false.

Note. A valid argument does not have any counter examples.

Classical logic idea:

- If an argument is not valid, there should be a counter example.

Definition 1.6. An argument is *sound* if it is valid and has only true premises.

Definition 1.7. The *syntax* is what you write which concerns itself to writing proofs and structure.

Definition 1.8. The *semantics* is what you mean which concerns itself to comparing to external.

Definition 1.9. An *atomic proposition* has no propositional substructure.

Note. Atomic propositions are imported from 'the world'.

Definition 1.10. We call a set of sentences *consistent* if there is an interpretation in which all members of that set are true.

2. Classical Propositional Logic

Classical propositional logic has several key concepts:

- Two truth values: True and False.
- Every proposition has a truth value.
- No proposition is both True and False.

$\neg$	$\wedge$	$\vee$	$\rightarrow$	$\leftrightarrow$
Negation	Conjunction	Disjunction	Implication	Biconditional

Definition 2.1. A *unary connective* joins two arguments.

Definition 2.2. A *binary connective* operates on one argument.

Definition 2.3. An *inductive formulae* is where each rule builds off of each other.

Definition 2.4. A *well-formed formula* (wff) is defined (inductively) by:

(1) Atomic formulae: α propositional variable, then α is a wff.
(2) Negation: α is wff, then $\neg\alpha$ is wff.
(3) Binary connective: if $\star$ is a binary connective, and α, β are wff then $(\alpha \star \beta)$ is a wff.

Note. Nothing formed without rules $1, 2, 3$ is a wff.

Definition 2.5. A *truth table* represents each case of an argument.

Example 2.6. The following is an example of using a truth table.

P	Q	$P \wedge \neg Q$
0	0	0
0	1	0
1	0	1
1	1	0

Definition 2.7. A formulae α is

- A tautology if true in all cases.
- A contradiction if false in all cases.
- Contingent if it is both true and false depending on the case.

Definition 2.8. A *literal* is a formulae which is either

- A single prop. variable.
- A negation of a single prop. variable.

Definition 2.9. A formulae is in *conjunctive normal form* if it is the conjunction of disjunctions of literals.

3. Natural Deduction

Note that Prawitz-style is used for Natural Deduction. For our purposes, we will define $\neg\alpha := \alpha \rightarrow \perp$ where $\perp$ is a propositional constant which is always false.

We say that Λ is the set of premises, D is an arbitrary deduction, and α is our conclusion. We write this as $\begin{smallmatrix}\Lambda\\ \alpha\end{smallmatrix} D$.

Definition 3.1. A *deduction* is defined inductively by:

(1) For each formulae α,
$$\alpha$$
is a proof with conclusion α, and assumptions $\{\alpha\}$.
(2) Every proper application of a *rule of inference* yields a proof.
(3) Anything which is not constructed finitely by rules (1) and (2) is not a proof.

Note. We say α is *derivable* from Λ if there is a deduction ${}^{\Lambda}_{\alpha}D$, and write $\Lambda \vdash \alpha$.

Definition 3.2. A wff α is a *theorem* if

$$\varnothing \vdash \alpha.$$

Definition 3.3. α and β are *logically equivalent* if $\alpha \vdash \beta$ and $\beta \vdash \alpha$. We (sometimes) write this as $\alpha \dashv\vdash \beta$.

3.1. **Rules of Inference.** In this section we will introduce the rules of inference. Each connective has two rules; an introduction rule and elimination rule.

Definition 3.4. A rule of inference is *derived* if every instance of it in a proof can be replaced with a proof which does not rely on the derived rule.

Firstly, the logical binary connective implication.

- The $\to I$ rule is given as:
  If we are given a deduction ${}^{\Lambda}_{\beta}D$, then

$$\dfrac{\begin{array}{c}\overline{\alpha}^{n}\\ {}^{\Lambda}D\\ \beta\end{array}}{\alpha \to \beta} \to I, n$$

  where assumption α is *discharged* at step n. Note that for discharging, you are able to discharge 0 or more times.

  Note. If $\Lambda \cup \{\alpha\} \vdash \beta$, then $\Lambda \setminus \{\alpha\} \vdash \alpha \to \beta$.

- The $\to E$ rule is given as:
  If we are given deductions ${}^{\Lambda_1}_{\alpha\to\beta}D_1$ and ${}^{\Lambda_2}_{\alpha}D_2$, then

$$\dfrac{\begin{array}{cc}\Lambda_1 & \Lambda_2\\ D_1 & D_2\\ \alpha \to \beta & \alpha\end{array}}{\beta} \to E$$

  where all assumptions are $\Lambda_1 \cup \Lambda_2$.

Example 3.5. In this example we will derive modus tollens (MT), which is use of contrapositive. Modus tollens is:

$$\dfrac{\alpha \to \beta \qquad \neg\beta}{\neg\alpha} MT$$

Deriving MT using our current rules of inference means we must prove

$$\alpha \to \beta, \neg\beta \vdash \neg\alpha.$$

This has the natural deduction:

$$\dfrac{\dfrac{\neg\beta \qquad \dfrac{\alpha \to \beta \qquad \overline{\alpha}^{1}}{\beta} \to E}{\bot} \to E}{\neg\alpha} \to I, 1$$

Example 3.6. In this example, we will derive the contrapositive. Contraposition is:

$$\dfrac{\alpha \to \beta}{\neg\beta \to \neg\alpha} Contrapose$$

Deriving contraposition using our current rules of inference means we must prove

$$\alpha \to \beta \vdash \neg\beta \to \neg\alpha.$$

This has the natural deduction:

$$\dfrac{\dfrac{\dfrac{\overline{\neg\beta}^{1} \qquad \dfrac{\alpha \to \beta \qquad \overline{\alpha}^{1}}{\beta} \to E}{\bot} \to E}{\neg\alpha} \to I, 2}{\neg\beta \to \neg\alpha} \to I, 1$$

Now, we will introduce the logical binary connective conjunction.

- The $\wedge I$ rule is given as:
  If we have deductions ${}^{\Lambda_1}_{\alpha} D_1$ and ${}^{\Lambda_2}_{\beta} D_2$, then

$$\dfrac{\begin{matrix}\Lambda_1 & \Lambda_2 \\ D_1 & D_2 \\ \alpha & \beta\end{matrix}}{\alpha \wedge \beta} \wedge I$$

  with assumptions $\Lambda_1 \cup \Lambda_2$.
- The $\wedge E$ rule is given as:
  If we have deduction ${}^{\Lambda}_{\alpha\wedge\beta} D$, then

$$\dfrac{\begin{matrix}{}^{\Lambda}D \\ \alpha \wedge \beta\end{matrix}}{\alpha} \wedge E$$

  and

$$\dfrac{\begin{matrix}{}^{\Lambda}D \\ \alpha \wedge \beta\end{matrix}}{\beta} \wedge E$$

We will now introduce the logical binary connective disjunction.

- The $\vee I$ is given as:
  If ${}^{\Lambda}_{\alpha} D$ is a deduction, then

$$\dfrac{\begin{matrix}{}^{\Lambda}D \\ \alpha\end{matrix}}{\alpha \vee \beta} \vee I$$

  and

$$\dfrac{\begin{matrix}{}^{\Lambda}D \\ \alpha\end{matrix}}{\beta \vee \alpha} \vee I$$

  where the assumptions do not change for any formula β.
- The $\vee E$ rule is given as:
  If we have deductions ${}^{\Lambda_1}_{\alpha\vee\beta} D_1$, ${}^{\Lambda_2}_{\alpha\to\gamma} D_2$, and ${}^{\Lambda_3}_{\beta\to\gamma} D_3$ then

$$\dfrac{\begin{matrix}\Lambda_1 & \Lambda_2 & \Lambda_3 \\ D_1 & D_2 & D_3 \\ \alpha \vee \beta & \alpha \to \gamma & \beta \to \gamma\end{matrix}}{\gamma} \vee E$$

  with assumptions $\Lambda_1 \cup \Lambda_2 \cup \Lambda_3$.

We now have all of the rules of inference for propositional minimal logic.

3.2. **Intuitionistic Absurdity.** If ${}^{\Lambda}_{\perp}D$ is a deduction, then

$$\dfrac{{}^{\Lambda}D \quad \perp}{\alpha}\,\perp$$

with assumptions Λ. Note that we sometimes write $\perp_i$ to distinguish itself. This rule is called Ex falso quodlibet (principle of explosion).

Minimal logic along with this rule is called intuitionistic logic. This is the foundation for constructive mathematics.

3.3. **Classical Absurdity.** If ${}^{\Lambda}_{\perp}D$ is a deduction, then

$$\dfrac{\overline{\neg\alpha}^{\,n} \quad {}^{\Lambda}D \quad \perp}{\alpha}\,\perp, n$$

is a deduction with assumptions $\Lambda \setminus \{\neg\alpha\}$. Note that we sometimes write $\perp_c$ to distinguish itself. This is 'reductio ad absurdum'.

Minimal logic along with this rule is called classical logic.

Example 3.7. The Law of Excluded Middle is:

$$\vdash \alpha \vee \neg\alpha.$$

Note that by Definition 3.2, LEM is a theorem in classical logic. The natural deduction is given as:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\overline{\neg(\alpha\vee\neg\alpha)}^{\,1} \quad \dfrac{\overline{\neg\alpha}^{\,2}}{\alpha\vee\neg\alpha}\vee I}{\perp_c}\to E}{\neg\neg\alpha}\to I, 2 \quad \dfrac{\dfrac{\overline{\neg(\alpha\vee\neg\alpha)}^{\,1} \quad \dfrac{\overline{\alpha}^{\,3}}{\alpha\vee\neg\alpha}\vee I}{\perp_c}\to E}{\neg\alpha}\to I, 3}{\perp_c}\to E}{\alpha\vee\neg\alpha}\,\perp_c, 1$$

4. Adequacy of the Propositional Rules

Definition 4.1. A *formal system* is

(1) *Sound* if it satisfies, allow deduction of only <u>valid</u> sequents. This is the property of *soundness*.
(2) *Complete* if it satisfies, allow deduction of all <u>valid</u> sequents. This is the property of *completeness*.

Definition 4.2. If from a derivation ${}^{\Sigma}_{\beta}D'$ we can obtain ${}^{\Lambda}_{\alpha}D$ by repeated (finite) applications of

(1) The rules of inference.
(2) Replacing any assumptions in ${}^{\Sigma}_{\beta}D'$ with a deduction concluding that assumption then we say ${}^{\Sigma}_{\beta}D'$ is a *subproof* of ${}^{\Lambda}_{\alpha}D$. If D and D' differ, then D' is a *proper subproof* of D.

Definition 4.3. Given a set of propositional variables $S = \{A_i \mid i < n\}$ a *valuation/validation function/assignment*

$$v : s \mapsto \{0, 1\}$$

assigns each A_i unique truth value. Given a formula α,

$$v(\alpha) = \begin{cases} 0 & \text{if } \alpha \text{ is false;} \\ 1 & \text{if } \alpha \text{ is true.} \end{cases}$$

Definition 4.4. The sequent $\Lambda \vdash \alpha$ is *valid* if for each assignment v, $v : x \mapsto 1$ whenever $v : \gamma \mapsto 1$ $\forall \gamma \in \Lambda$.

Theorem 4.5. *Natural deduction is sound (for propositional logic).*

Proof. Note that we will provide a partial proof.

Recall from Definition 3.1 that a derivation is one of:

- A single formula.
- An application of a rule of inference applied to an existing derivation.

Base case:

Deduction

$$\alpha$$

Assumptions are $\Gamma = \{\alpha\}$, conclusion is α. Trivially, $V(\alpha) = T$ whenever $V(\alpha) = T$.

Induction step:

Assume $\Gamma \vdash \beta$ is valid for any proper subproof ${}^{\Gamma}_{\beta}D$ in ${}^{\Gamma'}_{\beta}D'$. We need to prove that ${}^{\Gamma'}_{\beta}D'$ is valid. There are multiple cases for the rules of inference.

- $\to I$:

  Consider assumptions $\Gamma \setminus \{\alpha\}$ for

$$\dfrac{\begin{array}{c}\overline{\alpha}^{n} \\ {}^{\Gamma}D \\ \beta\end{array}}{\alpha \to \beta} \to I, n$$

□

Corollary 4.6. *All theorems of the propositional calculus* ($\vdash$) *are logical truths.*

Theorem 4.7. *The (classical) natural deduction calculus is complete.*

Note. This is a partial proof.

Proof.

$$\gamma_1, \ldots, \gamma_n \vdash \alpha$$

is valid then we use $\to I$ n times such that

$$\gamma_1 \to \ldots \to \gamma_n \to \alpha$$

is valid which then

$$\vdash \gamma_1 \to \ldots \to \gamma_n \to \alpha$$

is derivable. Then for use of $\to E$ n times

$$\gamma_1, \ldots, \gamma_n \vdash \alpha$$

is derivable.

If $\Gamma, \alpha \vdash \beta$ is valid, then $(\star)$ $\Gamma \vdash \alpha \to \beta$ is valid. Suppose $V(\gamma) = T$ for all $\gamma \in \Gamma$. Either $V(\alpha) = T$ or $V(\alpha) = F$. For if $V(\alpha) = F$, then certainly $V(\alpha \to \beta) = T$. Otherwise, by assumption $(\star)$,

$$V(\beta) = T$$

so $V(\alpha \to \beta) = T$. If $\Gamma \vdash \alpha \to \beta$ is derivable, then $\Gamma, \alpha \vdash \beta$ is derivable.

$$\dfrac{\begin{array}{c}\Gamma \\ D \\ \alpha \to \beta\end{array} \qquad \alpha}{\beta} \to E$$

□

5. First Order Logic (Predicate Logic)

Definition 5.1. The following are basic definitions for predicate logic:

(1) A *domain* is a collection of objects.
(2) An *n-place predicate* is a function which gives a truth value when supplied with n *names*.
(3) An *atomic formula* is an n-place predicate symbol applied to n names.
(4) A *propositional variable* is a 0-place predicate.

Example 5.2. M_x: x is a marine mammal. M is a 1-place predicate, x makes an object.

Two new logical symbols:

- $\exists$: "existential quantifier" (there exists)
- $\forall$: "universal quantifier" (for all or for every)

Definitions:

- Variables: $x_0, x_1, x_2, \ldots$ (infinitely many)
- Constants: $c_0, c_1, c_2, \ldots$
- Predicate Symbols: $R_0, R_1, R_2, \ldots$
- Function Symbols: $f_0, f_1, f_2, \ldots$
- Logical Operators: $\wedge, \vee, \rightarrow, \exists, \forall, \perp$ ($\neg\alpha := \alpha \rightarrow \perp$). Note that $\perp$ is now included in the definitions.
- Terms: variables and constants are terms. f is an n-arg function, and $t_0, \ldots, t_n$ are terms, then $f(t_0, \ldots t_n)$ is a term.

Corollary 5.3. *There are countably many proofs/theorems.*

Definition 5.4. A *wff* is defined as follows:

(1) $\perp$ is a wff.
(2) An n-arg predicate applied to n terms is an atomic formula (wff).
(3) If α, β are wff then $\alpha \rightarrow \beta$, $\alpha \wedge \beta$, $\alpha \vee \beta$, (and $\neg\alpha$) are wff.
(4) If α is a wff, and x is a variable, $\exists x\alpha$ and $\forall x\alpha$ are wff.

Note that the order of precedence is: $\forall, \exists, neg > \wedge > \vee > \rightarrow$.

5.1. Substitution.

- When substituting x with t in a term, every instance of x becomes t, and nothing else changes.
- Recurse into the arguments of a function
$$f(x, y, g(z, x))[x/t] = f(t, y, g(z, t))$$
such that $x \neq y$ and $x \neq z$.
- $\perp$ is unchanged (as are other 0-place predicates.)
- Recurse into the terms of any predicates.
- Recurse into the subformulae of propositional connectives. For example
$$P_x \rightarrow Q_{f(x,y)}[x/t] = P_t \rightarrow Q_{f(t,y)}$$
such that $x \neq y$.

So far, substitution is trivial. Write down t for every instance of x.

If the quantifying variable is to be substituted, do nothing. Otherwise, recurse into the subformulae.

$$\forall x(P_x \rightarrow R_{xy})[x/t] = \forall x(P_x \rightarrow R_{xy})$$

Idea from above: Not really the same x as

$$\forall x(P_x \rightarrow R_{xy}) \equiv \forall z(P_z \rightarrow R_{zy})$$

Yet,

$$\forall x(P_x \to R_{xy})[y/t] = \forall x(P_x \to R_{xt})$$

is an acceptable use of substitution.

By exclaiming 't is free for x', this is equivalent to saying 'it is okay to recurse into subformulae'.

For t to be free for x in $\forall y\alpha$, we require

(1) t is free for x in α, and
(2) $x \neq y$
(3) y does not appear in t.

For example, $f(x)$ is free for x in $\forall y P_{xy}$. But $f(y)$ is not free for x in $\forall y P_{xy}$.

Idea: t is not free for x in α if: substituting t in place of x would case a variable in t to become *bound*.

<u>Pitfall</u>: Do not confuse "free" and "free for" (okay to be replaced).

In formula

$$\forall x \exists y \Phi(x, y, z)$$

we say x and y are *bound*, and z is *free*.

To determine which variables are free/bound:

- Every variable in an atomic formulae is free.
- No variable in an atomic formulae is bound.
- For the propositional connectives: recurse to find free variables. For example, $P_x \to \forall y P_y$ has x free and y bound.

Any variables which are not free are bound.

Quantifiers:

- x is free in $\forall y\alpha$ if x free in α and $x \neq y$.
- x is bound in $\forall y\alpha$ if x is bound in α or $x = y$.

5.2. **Rules of Inference.** In this section we will add to our rules of inference for those of $\forall, \exists$ and $=$.

Firstly the universal quantifier.

- The $\forall I$ rule is given as:
  If $\overset{\Lambda}{\alpha[x/y]}D$ is a deduction and
  – y is not free in Λ, and
  – $x = y$ or y is not free in α
  – y is free for x in α
  then

$$\dfrac{\begin{matrix}\Lambda\\ D\\ \alpha[x/y]\end{matrix}}{\forall x\alpha}\ \forall I$$

  This is to prevent

$$\dfrac{P_y}{\forall x P x}\ \text{Invalid}$$

  as y must be arbitrary.
- The $\forall E$ rule is given as:
  If $\overset{\Lambda}{\forall x\alpha}D$, and t is free for x in α, then

$$\dfrac{\begin{matrix}\Lambda\\ D\\ \forall x\alpha\end{matrix}}{\alpha[x/t]}\ \forall E$$

  is a deduction with assumptions Λ.

Now, for the existential quantifier.

- The $\exists I$ is given as:
  If ${}^{\Lambda}_{\alpha[x/y]}D$ is a deduction and t is free for x in α, then
  $$\begin{array}{c} \Lambda \\ D \\ \alpha[x/t] \\ \hline \exists x\alpha \end{array} \exists I$$
  with assumptions Λ.
- The $\exists E$ is given as:
  If ${}^{\Lambda_1}_{\exists x\alpha}D_1$, ${}^{\Lambda_2}_{\alpha[x/y]\to\gamma}D_2$ are deductions, and y is not free in γ or in Λ_2, then
  $$\begin{array}{cc} \Lambda_1 & \Lambda_2 \\ D_1 & D_2 \\ \exists x\alpha & \alpha[x/y] \to \gamma \\ \hline \multicolumn{2}{c}{\gamma} \end{array} \exists E$$
  with assumptions $\Lambda_1 \cup \Lambda_2$.

As for identity, take 2-place infix predicate "=". We want the following axioms:

- $\forall x(x = x)$
- $\forall x \forall y(x = y \to P_x \to P_y)$

We do this by our rules of inference for identity.

- The $= I$ is given as:
  $$\overline{t = t}^{=I}$$
  is a deduction with no assumptions. This is reflexivity and known as the identity rule.
- The $= E$ is given as:
  If ${}^{\Lambda_1}_{\alpha[x/s]}D_1$, ${}^{\Lambda_2}_{s=t}D_2$, then
  $$\begin{array}{cc} \Lambda_1 & \Lambda_2 \\ D_1 & D_2 \\ \alpha[x/s] & s = t \\ \hline \multicolumn{2}{c}{\alpha[x/t]} \end{array} = E$$
  with assumptions $\Lambda_1 \cup \Lambda_2$.

6. Interpretations and Adequacy of Predicate Logic

6.1. **Interpretations of Predicate Logic.** The validation function for predicate logic:

$$V(R(t_1, \ldots, t_n)) = T \Leftrightarrow (I(t_1), \ldots, I(t_n)) \in I(R)$$

For example, suppose we have that

- Names:
  - Alice: a
  - Bob, Robert: b
  - Snowy: s
  - Tree: t
- Predicates:
  - Person: P
  - Dog: D
  - Loves: L

and that

- Alice and Bob love each other.
- Alice loves snowy.
- Snowy loves the tree.

We can then say that true statements are:

- $P_a \wedge D_s$
- $\neg D_t$
- $\exists x \exists y (L_{xy} \wedge L_{yx})$

and that a false statement is:

- $\forall x \forall y (L_{xy} \to \neg L_{yx})$

Therefore, using the validation function we have that

$$(I(a), I(b)) \in I(L).$$

Definition 6.1. $\Gamma \vdash \alpha$ is *valid* if *every interpretation* satisfying every $\gamma \in \Gamma$ also satisfies α.

Example 6.2. Our aim is to invalidate the following argument by showing an interpretation. There is something that, if it is a mammal, lives in the ocean. Therefore, there are ocean-dwelling mammals.

- M_x: x is a mammal.
- O_x: x lives in the ocean.

Suppose that

- $U_A = \{0, 1, 2\}$
- $I_A(M) = \varnothing$
- $I_A(O) = \{0\}$

For any $u \in U$,

$$V_{s[x \leftarrow u]}(M_x) = T \Leftrightarrow u \in I_A(M)$$

so, as $I_A(M) = \varnothing$,

$$V_{s[x \leftarrow u]}(M_x) = F$$

such that

$$V_{s[x \leftarrow u]}(M_x \to O_x) = T$$

and then certainly

$$V_s(\exists x (M_x \to O_x)) = T.$$

However,

$$V_{s[x \leftarrow u]}(M_x \wedge O_x) = F$$

so

$$V_s(\exists x (M_x \wedge O_x)) = F.$$

We have then shown an interpretation that invalidates the argument, thus

$$\exists x (M_x \to O_x) \nvdash \exists x (M_x \wedge O_x).$$

Example 6.3. We will show with an interpretation that the following argument is invalid. Everyone loves someone. Therefore, there is someone that everyone loves.

- L_{xy}: x loves y
- $U_A = \{0, 1\}$
- $I_A(L) = \{(0, 1), (1, 0)\}$

If $y = 0$, then $V_{s[y \leftarrow 0]}(\forall x L_{xy}) = F$ since $(0, 0) \notin I_A(L)$ (similarly for $y = 1$). So then certainly $V_s(\exists y \forall x L_{xy}) = F$.

We have that $V_s(\forall x \exists y L_{xy}) = T$ as

- $0 : 1$, L_{01}
- $1 : 0$, L_{10}

So then certainly

$$\forall x \exists y L_{xy} \nvdash \exists y \forall x L_{xy}$$

as required.

6.2. **Adequacy of Predicate Logic.** We will give an inductive proof in the style of soundness for propositional logic.

Inductive hypothesis: assume $\Gamma \vdash \beta$ is sound for every subproof ${}^{\Gamma}_{\beta}D$ of ${}^{\Gamma'}_{\alpha}D'$.

Check $\forall E$:

Take A such that $V_s(\gamma) = T$ for every $\gamma \in \Gamma$. By IH, $V_s(\forall x \alpha) = T$. Then for all $u \in U$

$$V_{s[x \leftarrow u]}(\alpha) = T.$$

t names some $u_t \in U$, so $V_s(\alpha[x/t]) = T$. t is free for x in α so

$$\begin{array}{c} \Gamma \\ D \\ \dfrac{\forall x \alpha}{\alpha[x/t]} \forall E \end{array}$$

holds.

Now, check $\forall I$:

Take A such that $V_s(\alpha) = T$ for all $\gamma \in \Gamma$. By the IH, $V_s(\alpha[x/y]) = T$. Let $u \in U$. Then since y does not occur (free) in γ, we could replace y with t, where $I_A(t) = u$. Then we have a proof of $\Gamma \vdash \alpha[x/t]$, which is also valid. $V_s(\forall \alpha) = T$ such that

$$\begin{array}{c} \Gamma \\ D \\ \dfrac{\alpha[x/y]}{\forall x \alpha} \forall I \end{array}$$

holds. Note that y is free for x in α and y is not free in Γ as required.

Now, check $\exists E$:

Now, check $\exists I$:

As for completeness,

- Completeness of propositional logic has a constructive proof. Algorithm for proving true propositional statements.
- Completeness of predicate logic (Kurt Gödel, 1929) is non-constructive. No algorithm (separate proof).

At least we have that predicate logic is both sound and complete.

7. Peano Arithmetic

Peano axioms contain a definition for natural numbers ($\mathbb{N}$), including equality. There are standard definitions for use with set theory. We will instead adopt the following (equivalent definitions). We add to the language:

- Constant $\dot{0}$,
- Unary function $\dot{S}$,
- Binary functions $\dot{+}$ and $\dot{*}$,
- and predicate $\dot{=}$ (2 place and infix).

The dots highlight that we're in the language of *Peano Arithmetic*.

7.1. **Axioms.**

(1) $\forall x\Big(\dot{0}\dot{\neq}\dot{S}(x)\Big)$ where $a\dot{\neq}b$ is $\neg(a\dot{=}b)$.

(2) $\forall x\forall y\Big(\dot{S}(x)\dot{=}\dot{S}(y) \to x\dot{=}y\Big)$

(3) $\forall x\big(x\dot{+}\dot{0}\dot{=}x\big)$

(4) $\forall x\forall y\Big(x\dot{+}\dot{S}(y)\dot{=}\dot{S}\big(x\dot{+}y\big)\Big)$

(5) $\forall x\big(x\dot{*}\dot{0}\dot{=}\dot{0}\big)$

(6) $\forall x\forall y\Big(x\dot{*}\dot{S}(y)\dot{=}x\dot{*}y\dot{+}x\Big)$

(7) $\alpha[x/\dot{0}] \to \Big(\forall x\Big(\alpha \to \alpha[x/\dot{S}(x)]\Big) \to \forall x\alpha\Big)$

This axiom gives us the principle of induction. It says that every natural number is either zero or a successor, i.e. every natural number is reachable from zero by taking successors finitely many times.

Kurt Gödel's incompleteness theorem:

First order logic + PA is incomplete. See *continuum hypothesis*[1]: is there an infinity between $|\mathbb{N}|$ and $|\mathbb{R}|$?

Definition 7.1. We define the numbers in PA as

- $\dot{1} := \dot{S}\big(\dot{0}\big)$
- $\dot{2} := \dot{S}\big(\dot{1}\big) \equiv \dot{S}\dot{S}\dot{0}$
- and so on.

Example 7.2. We will prove

$$PA \vdash \forall x\big(\dot{0}\dot{+}x\dot{=}x\big)$$

by induction. We will forgo using dots for simplicity. Note that

$$\alpha \equiv 0 + x = x.$$

Base case:

$$\dfrac{P3}{0+0=0}\,\forall E$$

Induction step:

$$\dfrac{\dfrac{\dfrac{\dfrac{P4}{0+S(x)=S(0+x)}\,\forall E(\times 2) \qquad \dfrac{}{0+x=x}^{1}}{0+S(x)=S(x)}\,=E}{0+x=x \to 0+S(x)=S(x)}\,\to I,1}{\forall x(0+x=x \to 0+S(x)=S(x))}\,\forall I$$

For which then:

$$\dfrac{\dfrac{P7 \qquad \text{Base case}}{\forall x(\alpha \to \alpha[x/S(x)]) \to \forall x\alpha}\,\to E \qquad \text{Ind. step}}{\forall x\alpha}\,\to E$$

Note. We could go on to prove and define many things such as

- Define: $x < y := \exists z(x + S(z) = y)$

8. Kripke Models

8.1. **Minimal and Intuitionistic Logic.** Recall:

$$\dfrac{\begin{matrix}\Gamma\\ D\end{matrix}}{\alpha}\,\bot$$

and

$$\overline{\neg\alpha}^{n}$$
$$\Gamma D$$
$$\frac{\perp}{\alpha}\perp, n$$

- A deduction which uses neither rule is a deduction in *minimal logic*.
- Using only the intuitionistic absurdity is called *intuitionistic logic*.
- Using the classical absurdity is called *classical logic*.

We say that

- $\Gamma \vdash_m \alpha$ refers to a minimal logic derivation,
- $\Gamma \vdash_i \alpha$ refers to an intuitionistic logic derivation,
- and $\Gamma \vdash_c \alpha$ refers to a classical logic derivation.

This gives us a hierarchy of

$$\textbf{Minimal proof} \Rightarrow \textbf{Intuitionistic proof} \Rightarrow \textbf{Classical proof}.$$

For intuitionistic logic, this informs constructive logic. We should be able to construct any object that we discuss and claim to exist. For example,

Proposition 8.1.

$$\exists x \exists y (x \notin \mathbb{Q} \wedge y \notin \mathbb{Q} \wedge x^y \in \mathbb{Q}).$$

Note. This proof will make use of LEM by the point of view of a classical mathematician.

Proof. Consider $\sqrt{2} \notin \mathbb{Q}$. Thus, let $x = \sqrt{2}$ and $y = \sqrt{2}$. If $\sqrt{2}^{\sqrt{2}} \in \mathbb{Q}$, our proof is complete. If not, then let $x = \sqrt{2}^{\sqrt{2}}$ and $y = \sqrt{2}$ such that $x^y = 2 \in \mathbb{Q}$. So, $\exists x \exists y (x \notin \mathbb{Q} \wedge y \notin \mathbb{Q} \wedge x^y \in \mathbb{Q})$ as required. □

With intuitionistic logic, this proof does not hold due to our semantics. We cannot have LEM, so interpretations won't work.

8.2. **Kripke Models.**

Definition 8.2. A *partial order* on set X is a reflexive ($x \preccurlyeq x$), transitive ($x \preccurlyeq y \wedge y \preccurlyeq z \to x \preccurlyeq z$), and antisymmetric relation ($x \preccurlyeq y \wedge y \preccurlyeq x \to x = y$).

Example 8.3. The following is an example of Definition 8.2.

$$\begin{tikzcd}
 & & d \\
 & & c \arrow[u, "c \preccurlyeq d"] \\
b & & \\
 & a \arrow[ul, "a \preccurlyeq b"] \arrow[uur, "a \preccurlyeq c"] &
\end{tikzcd}$$

Definition 8.4. A *Kripke model* is a partially ordered set $(X, \preccurlyeq)$ and function Σ mapping each vertex k (world) to a subset of propositional variables such that

$$k \preccurlyeq l \Rightarrow \Sigma(k) \subseteq \Sigma(l).$$

ie Σ is monotone. Also assume $\perp \notin \Sigma(k)$.

Example 8.5. An example of a Kripke model is given below.

$$\begin{tikzcd}
 & & d \\
 & & c \arrow[u, "P,Q"] \\
b & & \\
 & a(P) \arrow[uul, "R,P"] \arrow[uur, "P,Q"'] &
\end{tikzcd}$$

We will add onto this example later to clarify.

Definition 8.6. If $K = ((x, \preccurlyeq), \Sigma)$ is a Kripke model, define $k \Vdash P \Leftrightarrow p \in \Sigma(k)$.

Note. We say informally that k *forces* P is $k \Vdash P$.

Now,

- $k \Vdash \alpha \wedge \beta$ if $k \Vdash \alpha$ and $k \Vdash \beta$
- $k \Vdash \alpha \vee \beta$ if $k \Vdash \alpha$ or $k \Vdash \beta$
- $k \Vdash \alpha \to \beta$ if for every l such that $k \preccurlyeq l$, if $l \Vdash \alpha$ then $l \Vdash \beta$

Lemma 8.7. *Forcing is also monotone.*

Example 8.8. We will further the example set in Example 8.5.

$$\begin{tikzcd}
 & & d(P,Q) \\
b(P,R) & & c(P,Q) \arrow[u] \\
 & a(P) \arrow[ul] \arrow[ur] &
\end{tikzcd}$$

We then say that

- $a \Vdash P$
- $c \Vdash P \wedge Q$
- $a \Vdash Q \to P$
- $a \nVdash P \wedge Q$

Definition 8.9. We say that $K = ((X, \preccurlyeq), \Sigma)$ forces α if $k \Vdash \alpha$ for all $k \in X$.

Definition 8.10. If $k \Vdash \alpha$ for every k such that ($k \Vdash \gamma$ for all $\gamma \in \Gamma$), then we write $\Gamma \Vdash_k \alpha$.

Theorem 8.11. *Intuitionistic logic is sound for Kripke semantics. If* $\Gamma \vdash_i \alpha$ *then* $\Gamma \Vdash_k \alpha$ *for all* k.

Note. Completeness also holds.

Example 8.12. In this example we provide another kripke model.

$$\begin{tikzcd}
 & l(P, P \vee \neg P, \neg\neg P) \\
 & k(\neg\neg P) \arrow[u]
\end{tikzcd}$$

Thus,

- $k \nVdash P$
- $k \nVdash \neg P$
- $k \nVdash P \vee \neg P$
- Hence, $\nvdash_i P \vee \neg P$

Example 8.13. In this example we provide another kripke model.

$$\begin{tikzcd} b(Q) & & c(P) \\ & a \arrow[ul] \arrow[ur] & \end{tikzcd}$$

Thus,

- $b \nVdash Q \to P$
- $c \nVdash P \to Q$
- $a \nVdash (Q \to P) \vee (P \to Q)$

An intuitionistic theorem is (Paradox of Entailment)

$$\vdash_i (\neg P \wedge P) \to Q.$$

A counter example using a minimal logic Kripke model:

$$\text{k}$$

for which $k \Vdash P$ and $k \Vdash \bot$ such that $k \Vdash P \to \bot$. This is not an intuitionistic Kripke model as we are not allowed to have $\bot$ in any world.

9. Applications

We will now describe applications of what we have learned. The programming language Agda makes use of intuitionistic logic. It is based on Zhaohui Luo's unified theory of dependent types (UTT). This is a type theory similar to Martin-Löf type theory (intuitionistic type theory).[2] Agda is a functional programming language implemented by Haskell.

Agda is a

- proof assistant,
- functional programming language,
- based on intuitionistic (constructive) type theory,
- and propositions as types programming language.

We will be showing code that could be run by Agda to define the naturals $\mathbb{N}$.
– Define $\mathbb{N}$ as either a zero or a successor:
data $\mathbb{N}$: proposition where
zero : $\mathbb{N}$
suc : $(n : \mathbb{N}) \to \mathbb{N}$
– View this as an argument from two premises n and m
– to the conclusion that there exists naturals.
infixl 6 +
$+ : (nm : \mathbb{N}) \to \mathbb{N}$
zero + m = m
suc $n + m =$ suc $(n + m)$
– $n + m = ?$

The file agda-test.agda provides the rest of the demo.

By using a programming language based on constructive logic, we can prove that functions do what we intend them to do. Huge area of research:

(1) Defining algorithms within programming languages which have these types of proofs in them in order to make sure their algorithms are correct.
(2) For example, the Paris metro: the routing algorithm for trains in the Paris metro is defined in an constructive type theoretic programming language such that we can mathematically prove that no two trains will be routed on the same track.

Note that beyond a programming language such as Agda, formal logic has many applications. While we will not discuss them all, formal logic has many connections to computability. This is in relation to classes of complexity such as the P vs. NP problem.

10. References

[1]Wikipedia (2020, August 25). Continuum hypothesis. *Wikipedia, the free encyclopedia.* Retrieved from
`https://en.wikipedia.org/wiki/Continuum_hypothesis`

[2]Wikipedia (2020, August 25). Agda (programming language). *Wikipedia, the free encyclopedia.* Retrieved from
`https://en.wikipedia.org/wiki/Agda_(programming_language)`